

Instructions

Lab Repo and setup instructions: <https://github.com/MarazMia/Web-Application-Security-Lab/tree/main>

Lab Task related instructions: https://github.com/MarazMia/Web-Application-Security-Lab/blob/main/lab1_api_security/vulnerable_api/LAB_API_SECURITY_README.md

Results

1. JWT Secret Management

Hardcoded token accessible by anyone viewing the source code in `app.py`.

```
10 # Dynamically set the database path relative to the location of app.py
11 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
12 DB = os.path.join(BASE_DIR, "vulnmart.db")
13
14 # TODO (1) - JWT Secret Management
15 # This is a weak, hardcoded, guessable secret - anyone reading this file
16 # could forge valid tokens with it.
17 #
18 # >>> Replace the line below with:
19 #     JWT_SECRET = os.environ.get("VULNMART_JWT_SECRET", secrets.token_hex(32))
20 JWT_SECRET = "vulnmart-secret-123"
21 JWT_ALGO = "HS256"
22 JWT_EXPIRATION_SECONDS = 3 * 60 # token lifetime to use once TODO (3) is fixed
```



The secure application obtains the JWT secret from the `VULNMART_JWT_SECRET` environment variable and uses a cryptographically secure random fallback when the variable is not configured.

2. Restrictive CORS

Vulnerable App is too permissive, allowing cross-origin requests. The following headers appear:

- Access-Control-Allow-Origin: <http://evil.example>
- Access-Control-Allow-Credentials: true

With the Secure App, requests from an untrusted origin such as `http://evil.example` do not receive an `Access-Control-Allow-Origin` response header granting access. Requests from an explicitly allowed origin, such as `http://127.0.0.1:5001`, receive the appropriate `Access-Control-Allow-Origin` header.

3. JWT Expiration

With the vulnerable application, a 200 response (OK) is always returned.

```
(.venv) matthewlove@Matthews-MacBook-Pro-2 vulnerable_api % curl -i
"http://127.0.0.1:5000/api/users/1" \
-H "Authorization: Bearer $token"

HTTP/1.1 200 OK
Server: Werkzeug/3.0.4 Python/3.12.0
Date: Thu, 01 Oct 2026 11:20:47 GMT
Content-Type: application/json
```

```
Content-Length: 286
Connection: close
```

The secure application token expires after 3 minutes. Trying to invoke the token after that 3 minute window returns a 401 response (unauthorized), since the token is expired.

```
(.venv) matthewlove@Matthews-MacBook-Pro-2 secure_api % curl -i
"http://127.0.0.1:5001/api/users/1" \
-H "Authorization: Bearer $token

HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.0.4 Python/3.12.0
Date: Thu, 01 Oct 2026 11:18:51 GMT
Content-Type: application/json
Content-Length: 25
Access-Control-Allow-Origin: http://127.0.0.1:5001
Vary: Origin
Connection: close
```

4. Rate Limiting

Vulnerable app test results (1000 "200" responses)

```
=====
RATE LIMIT TEST RESULTS
=====
Total requests: 1000
401 responses: 0
200 responses: 1000
429 responses: 0
Other responses: 0
Elapsed time: 80.57 seconds
=====
```

Secure app test results (Only first 5 responses were "200". The next 995 were all "429"). It throttles repeated authentication attempts after 5 requests per minute.

```
=====
RATE LIMIT TEST RESULTS
=====
Total requests: 1000
401 responses: 0
200 responses: 5
429 responses: 995
Other responses: 0
Elapsed time: 1.54 seconds
=====
```

5. Registration Mass Assignment

Vulnerable app allows API call to POST whatever values it wants for balance and is_admin. The logged in account shows

User Profile

Account information

Vulnerable App

Profile Information

User ID	6
Username	mallory1
Email	mallory1@example.com
SSN	
Balance	\$999999
Administrator	Yes

those values were actually set:

The Secure app, however, does not allow balance and is_admin to be set by the API at all. A logged in account shows

User Profile

Account information

Secure App

Profile Information

User ID	5
Username	mallory1
Email	mallory1@example.com
SSN	
Balance	\$
Administrator	No

that they were properly set to 0:

The secure app (and also the vulnerable app in this case) also handles invalid input for other fields, such as the username. An invalid username will return "username cannot be empty" or "username already exists".

6. Profile BOLA / IDOR and Excessive Data Exposure

Vulnerable app exposes everything about Bob to Alice:

- ssn: 222-33-4444
- balance: 40.0
- password_hash: ...
- is_admin: 0

Secure app exposes nothing about Bob to Alice, and instead returns a 403 "forbidden" response.

- ~~ssn~~
- ~~balance~~
- ~~password_hash~~
- ~~is_admin~~

7. Update User: Ownership and Property-Level Authorization

	Vulnerable App	Secure App
Alice updating her own account	✅ Allows update on all fields	— Allows update only on unprotected fields (the protected fields, <code>balance</code> and <code>is_admin</code> , are not updated)
Alice updating Bob's account	✅ Allows update on all fields	❌ 403 Forbidden

8. BOLA: Access Another User's Orders

	Vulnerable App	Secure App
Alice viewing her own orders	✅ Shows all of her own orders	✅ Shows all of her own orders
Alice viewing Bob's orders	✅ Shows all of Bob's orders	❌ 403 Forbidden

9. BOLA: Access Another User's Individual Order

Same behavior as accessing all orders

	Vulnerable App	Secure App
Alice viewing one of her own orders directly	✅ Shows her order	✅ Shows her order
Alice viewing one of Bob's orders directly	✅ Shows Bob's orders	❌ 404 Not Found

10. Missing Function-Level Authorization

Vulnerable app allows any authenticated user to call an admin endpoint.

Secure app only allows an authenticated *admin* user to call an admin endpoint. A non-admin receives a 403 Forbidden response.

11. Bonus — SQL Injection Products API/Page

Yes, with the vulnerable app, the query is altered by the SQL injection and returns rows that would never be returned by a normal search (which in this case, happens to be all the user data!).

The secure app, nothing is returned except a 401 Unauthorized response because the search input is actually treated like a search input, and not escaped SQL

Interestingly, when using the VulnMart Product / User Search page, both the Vulnerable and Secure apps just return the following when searching `' OR '1'='1`

```
{
  "error": "unauthorized"
}
```

12. Bonus — API Versioning and Inventory

The Secure app has a defined version: `v1`

Health check response:

```
{"status":"ok","version":"v1"}
```

The application simply records the version in the health response. There is no `/v1/` prefix in the API. Version definition:

```
def health():  
    return jsonify({"status": "ok", "version": "v1"})
```

- Version exposed: `v1`
- Version location: `health` endpoint
- Health endpoint: `GET /api/health`

The Vulnerable app has no version definition at all.

API Inventory

Endpoint	Method	Authentication	Authorization
<code>/api/login</code>	POST	No	Public
<code>/api/register</code>	POST	No	Public
<code>/api/users/<id></code>	GET	Yes	Object owner
<code>/api/users/<id></code>	PUT	Yes	Object owner + property restrictions
<code>/api/users/<id>/orders</code>	GET	Yes	Object owner
<code>/api/orders/<id></code>	GET	Yes	Order owner
<code>/api/admin/users</code>	GET	Yes	Administrator only
<code>/api/products</code>	GET	Yes	Authenticated user
<code>/api/health</code>	GET	No	Public

Submission Questions

Q1 - Why is authentication different from authorization?

Authentication ensures that you are permitted to act as an actor. Authorization ensures that you are permitted to act on a resource.

Q2 - Which TODOs implement object-level authorization?

TODOs 6, 7, 8, and 9.

Q3 - Which TODOs prevent client-controlled sensitive properties?

TODOs 5 and 7.

Q4 - Why does parameterized SQL prevent the SQL injection checkpoint from changing query structure?

The user input is treated as data rather than executable SQL syntax, so characters such as quotes and SQL operators cannot change the structure or logic of the query.

Q5 - Why can preventing SQL injection alone be insufficient if a frontend renders untrusted API values using unsafe HTML insertion?

Preventing SQL injection protects the database query, but it does not automatically make returned data safe to render in a browser. If the frontend inserts untrusted API values using unsafe HTML insertion (like `innerHTML`), attacker-controlled content could be interpreted as HTML or script. The frontend must therefore safely encode or render untrusted values as text.

Q6 - Why is returning a full database row dangerous even when access to the endpoint requires authentication?

Because the user may not be authorized to see all of the elements in that row.

Q7 - Why is an authenticated user not automatically authorized to access an administrator endpoint?

Authentication verifies who the user is, while authorization determines what they can access. An authenticated user is not automatically an administrator.

Q8 - What behavior changed after rate limiting was implemented?

The Secure app only allowed the first 5 attempts before returning `429 Too Many Requests` for the remaining 995 requests. This prevents an attacker from gaining login credentials through brute force attacks. It also could prevent denial of service attacks by throttling the number of costly requests the server performs.

Q9 - What is the purpose of token expiration?

It limits the time a stolen token is useful to attackers.

Q10 - What information should be maintained in an API inventory?

An API inventory should record each active endpoint, the HTTP methods it accepts, whether authentication is required, and what authorization rules apply. It can also document the API version, endpoint purpose, required parameters, and other relevant security requirements.

AI Accreditation

- ChatGPT for debugging and help with questions: <https://chatgpt.com/share/6abeab17-4b7c-83e9-8522-447801c5652c>
- Copilot in VS Code for debugging